

Relatório de Projecto

Programação Aplicada

Avaliação Periódica 2

Autor(es):

Mariana Magalhaes a2022147454
Dinis Myroshnyk a2021157297

Data: 28/06/20

Resumo

O presente relatório descreve o desenvolvimento de uma aplicação de gestão de reparações de equipamentos, no âmbito da unidade curricular de Programação Aplicada do curso de Licenciatura em Engenharia Informática. A aplicação foi desenvolvida em linguagem Java, seguindo o paradigma de Programação Orientada a Objetos, com recurso a uma base de dados NoSQL SurrealDB para persistência de dados. O sistema permite gerir utilizadores com diferentes perfis de acesso (administrador, cliente e funcionário), equipamentos, pedidos de reparação e respetivos processos de intervenção. Foram implementados os requisitos propostos, incluindo autenticação de utilizadores, gestão de equipamentos e reparações, sistema de notificações, registo de ações através de logs automáticos via triggers na base de dados, envio de emails aquando do registo de novos utilizadores, e suporte a fotografias de perfil para cada utilizador, com recurso a uma imagem por defeito caso não seja definida. A interface com o utilizador foi desenvolvida recorrendo à biblioteca JavaSwing, proporcionando uma interface gráfica intuitiva e de fácil utilização.

Palavras-chave

Base de dados, gestão de reparações, Java, JDBC, logs, Programação Orientada a Objetos, SurrealDB, triggers

Índice

Resumo.....	2
Lista de Figuras	4
Lista de Tabelas.....	4
Lista de Acrónimos.....	5
1. Introdução.....	6
2. Objectivos e Metodologias	8
2.1. Ferramentas e Tecnologias	8
2.2. Planeamento	9
3. Trabalho Desenvolvido	10
3.1. Requisitos Implementados.....	10
3.2. Classes e <i>Packages</i>	10
3.3. Algoritmos	13
3.4. Estruturas de Dados	17
3.5. Armazenamento de Dados.....	19
3.6. Procedimentos de Teste.....	22
4. Conclusões	23
4.1. Forças	23
4.2. Limitações	23
4.3. Trabalho Futuro.....	23
5. Referências.....	24
6. Anexos	25

Lista de Figuras

Figura 1 - Diagrama ER..... 19

Lista de Tabelas

Tabela 1 - fases do projeto 9

Lista de Acrónimos

API	Application Programming Interface
BD	Base de Dados
ER	Modelo Entidade-Relacionamento
JDBC	Java Database Connectivity
JDK	Java Development Kit
LEI	Licenciatura em Engenharia Informática
NoSQL	Not Only SQL
OOP	Object-Oriented Programming (Programação Orientada a Objetos)
POO	Programação Orientada a Objetos
SKU	Stock Keeping Unit
SoA	State of the Art
SQL	Structured Query Language
UML	Unified Modeling Language

1. Introdução

No contexto atual, a gestão eficiente de processos de reparação de equipamentos representa um desafio para empresas que prestam serviços de assistência técnica. A ausência de um sistema centralizado dificulta o acompanhamento dos pedidos de reparação, a comunicação entre os diferentes intervenientes e o controlo do estado dos processos em curso.

O presente trabalho surge no âmbito da unidade curricular de Programação Aplicada, do curso de Licenciatura em Engenharia Informática, e procura o desenvolvimento de uma aplicação de gestão de reparações de equipamentos. A aplicação vem auxiliar o registo de pedidos de reparação e o processo de intervenção de um departamento de apoio ao cliente, permitindo a interação entre três tipos de utilizadores: administradores, clientes e funcionários.

A solução proposta foi desenvolvida em linguagem Java, seguindo o paradigma de Programação Orientada a Objetos, recorrendo a uma base de dados SurrealDB para a persistência de dados. A interface com o utilizador foi implementada recorrendo à biblioteca JavaSwing, proporcionando uma interface gráfica intuitiva e de fácil utilização.

Relativamente às fotografias de perfil dos utilizadores, optou-se por guardar as imagens no mesmo servidor onde corre a base de dados SurrealDB, sendo armazenado na base de dados apenas o nome do ficheiro. Esta decisão foi tomada para garantir que a aplicação funciona corretamente independentemente da máquina onde é executada, evitando erros decorrentes da ausência de ficheiros locais em diferentes ambientes.

No que diz respeito ao sistema de envio de emails, este foi implementado com recurso à biblioteca Jakarta Mail e envia uma notificação para um endereço de email predefinido aquando do registo de novos utilizadores. Por questões de configuração do serviço Gmail, que requer autenticação de dois fatores para permitir o envio de emails por aplicações externas, optou-se por utilizar um endereço de teste criado para o efeito. Esta abordagem permitiu validar o funcionamento do requisito, ficando a configuração com um endereço real dependente do ambiente de produção.

Para a geração de extratos em PDF, foi utilizada a biblioteca Apache PDFBox. Esta funcionalidade, correspondente ao requisito G6, permite imprimir um extrato das ações realizadas num processo de reparação, incluindo as datas e os utilizadores que realizaram cada ação. O documento é gerado a partir dos logs associados à reparação e formatado automaticamente pela biblioteca, sendo posteriormente disponibilizado ao utilizador para impressão ou arquivo.

Caso existam dificuldades na execução do projeto, o repositório do código fonte encontra-se disponível no GitHub, acompanhado de instruções para descarregar e executar o ficheiro JAR da aplicação. Adicionalmente, encontra-se em anexo uma pasta contendo o diagrama entidade-relação da base de dados e a listagem detalhada dos requisitos implementados. O diagrama é apresentado em anexo por a sua dimensão não permitir uma visualização adequada no corpo do relatório, enquanto a listagem de requisitos foi elaborada como documento de apoio ao desenvolvimento e aproveitada para integrar o relatório como referência complementar.

O presente documento está organizado em cinco capítulos. Após esta introdução, o capítulo 2 descreve os objetivos e metodologia adotada, o capítulo 3 detalha o trabalho desenvolvido e os requisitos implementados, e o capítulo 4 apresenta as conclusões e possíveis melhorias futuras.

2. Objectivos e Metodologias

O principal objetivo deste trabalho é a migração da aplicação de gestão de reparações de equipamentos desenvolvida no projeto anterior para uma interface gráfica, recorrendo à biblioteca JavaSwing, mantendo a lógica de negócio e a base de dados SurrealDB já implementadas. A aplicação deve cumprir os novos requisitos definidos no enunciado, nomeadamente a adição de fotografias de perfil, envio de emails de registo, sistema de notificações em destaque após autenticação, impressão de extratos de ações, barras de deslocamento nas listagens, tooltips de ajuda nos formulários e componentes gráficos adequados a cada tipo de informação.

A metodologia adotada foi iterativa e colaborativa, dividida nas seguintes fases.

Em primeiro lugar, ambos os elementos do grupo analisaram o enunciado e identificaram as alterações necessárias face ao projeto anterior, nomeadamente a substituição da interface em modo texto por uma interface gráfica. De seguida, um dos elementos iniciou o desenvolvimento da nova interface gráfica, estabelecendo os primeiros ecrãs e definindo a estrutura base a seguir. O outro elemento procedeu à refatoração do código desenvolvido, garantindo consistência e alinhamento com a estrutura do projeto anterior. A partir do momento em que a estrutura base ficou consolidada, a aplicação do mesmo padrão aos restantes ecrãs e modelos tornou-se significativamente mais rápida, permitindo concluir o desenvolvimento de forma eficiente com a divisão de tarefas entre os dois elementos do grupo.

2.1. Ferramentas e Tecnologias

No desenvolvimento do projeto foram utilizadas as seguintes ferramentas e tecnologias:

- **Java 25:** linguagem de programação principal, seguindo o paradigma de Programação Orientada a Objetos.
- **SurrealDB:** base de dados NoSQL utilizada para a persistência de todos os dados da aplicação, tirando partido das suas funcionalidades avançadas como triggers e funções nativas.
- **Driver oficial SurrealDB para Java:** biblioteca disponibilizada pelo SurrealDB para estabelecer a ligação entre a aplicação Java e a base de dados.
- **Visual Studio Code:** ambiente de desenvolvimento integrado utilizado por ambos os elementos do grupo.
- **PlantUML:** ferramenta utilizada para a elaboração dos diagramas UML, nomeadamente o diagrama de classes.
- **Sistemas operativos:** o desenvolvimento decorreu em dois sistemas operativos distintos, Windows 11 e Linux (CentOS), garantindo a compatibilidade da aplicação em ambientes diferentes.
- **JavaSwing:** biblioteca gráfica do Java utilizada para o desenvolvimento de toda a interface gráfica da aplicação.
- **Jakarta Mail:** biblioteca utilizada para o envio de emails, nomeadamente para notificar os utilizadores aquando do registo na aplicação.
- **Apache PDFBox:** biblioteca utilizada para a geração de extratos em PDF com o histórico de ações de cada processo de reparação.

2.2. Planeamento

O desenvolvimento do projeto não seguiu um planeamento formal rígido, tendo evoluído de forma iterativa ao longo de aproximadamente duas semanas, com trabalho realizado principalmente durante as tardes e noites.

A divisão de tarefas foi a seguinte: ambos os elementos participaram na análise do enunciado e identificação dos novos requisitos face ao projeto anterior. O desenvolvimento foi posteriormente dividido, ficando um elemento responsável pelo início da implementação da interface gráfica em JavaSwing, estabelecendo os primeiros ecrãs e a estrutura base a seguir. O outro elemento procedeu à refatoração e consolidação dessa estrutura base, garantindo consistência ao longo de todo o código, tendo também desenvolvido outras funcionalidades. A partir do momento em que a estrutura base ficou consolidada, a aplicação do mesmo padrão aos restantes ecrãs foi dividida entre os dois elementos, o que permitiu acelerar significativamente o ritmo de desenvolvimento.

Cada fase do projeto consumiu aproximadamente uma tarde ou noite de trabalho, conforme descrito na tabela seguinte:

Tarefa	Responsável	Tempo Estimado
Análise do enunciado e diagrama de classes	<i>Ambos</i>	<i>1 tarde</i>
Início do desenvolvimento da interface gráfica	<i>Mariana</i>	<i>3 tarde</i>
Refatoração da estrutura base	<i>Dinis</i>	<i>2 tarde e 1 noite</i>
Implementação dos restantes ecrãs e modelos	<i>ambos</i>	<i>3 tarde e 1 noite</i>
Testes e correções	<i>Dinis</i>	<i>3 tarde</i>
JavaDoc	<i>--</i>	<i>1 tarde</i>
Relatório	<i>Mariana</i>	<i>1 tarde</i>

Tabela 1 - fases do projeto

3. Trabalho Desenvolvido

3.1. Requisitos Implementados

A listagem detalhada dos requisitos implementados encontra-se em anexo. O anexo inclui dois conjuntos de requisitos:

- Os requisitos do projeto 1, onde alguns se encontram riscados por não serem aplicáveis no contexto do projeto 2, nomeadamente os relacionados com a interface em modo texto.
- Os requisitos do projeto 2, com indicação do estado de implementação de cada um.

3.2. Classes e *Packages*

3.2.1. Package main

Este é o package principal e o ponto de entrada da aplicação. Contém as classes centrais que orquestram o funcionamento do sistema.

- **App.java:** A classe que inicia a aplicação.
 - `main(String[] args)`: O método principal que agenda a inicialização da interface gráfica (GUI) na Event Dispatch Thread (EDT) do Swing.
 - `start()`: Cria a janela principal (JFrame), inicializa a classe State e gere o fluxo de arranque: verifica se o ficheiro de configuração existe (DBConnectionState), conecta-se à base de dados e, por fim, verifica se existe um administrador para decidir entre o ecrã de configuração inicial (FirstInitState) ou o de login (SignInState).
- **DatabaseManager.java:** Uma classe Singleton que centraliza toda a comunicação com a base de dados SurrealDB. É a única classe no projeto que executa queries diretamente, abstraindo a lógica de acesso aos dados do resto da aplicação.
 - `getInstance()`: Implementa o padrão Singleton para garantir que existe apenas uma instância desta classe.
 - `connect()`: Estabelece a ligação à base de dados usando as credenciais geridas pelo PropertiesManager.
 - `saveUser(User user)` / `updateUser(User user)`: Métodos que gerem a persistência de utilizadores. Utilizam transações para garantir a consistência dos dados ao criar ou atualizar as várias tabelas relacionadas (user, registrable_user, employee/client).
 - `getRepairs(...)` / `fetchRepair(...)`: Métodos para consultar reparações.
 - `updateRepairState(..., Double cost)`: Um método crucial que atualiza o estado de uma reparação. É um bom exemplo de um método não óbvio, pois para além de alterar o estado, também pode atualizar o custo (se

fornecido) e o campo de observações, e ainda despoletar o envio de notificações a utilizadores relevantes.

- `userFromValue(Value element)` / `repairFromValue(Value element)`: Métodos privados que convertem a resposta da base de dados (um objeto `Value`) para os modelos Java correspondentes (`User`, `Repair`). Esta conversão é fundamental para a leitura de dados.
- **PropertiesManager.java**: Classe responsável por ler e escrever propriedades de configuração no ficheiro `db/config.properties`. Isola a gestão de configurações como URLs, tokens de API e credenciais da base de dados.
 - `getProperty(String key)`: Obtém o valor de uma propriedade.
 - `saveFile()`: Guarda as propriedades no ficheiro.
- **ViewManager.java**: Classe de utilidade que ajuda a criar componentes de UI consistentes, como painéis com títulos (`titledPanel`), garantindo uma aparência uniforme em toda a aplicação.

3.2.2. Package models

Este package contém as classes de modelo (o "M" do padrão MVC). São objetos simples (POJOs) que representam as entidades de dados do sistema, como utilizadores, reparações e equipamentos.

- **User.java**: Classe abstrata que serve de base para todos os tipos de utilizador. Define atributos comuns como `name`, `username`, `email`, `status`, etc. Utiliza o padrão de projeto Builder para a criação de objetos, o que facilita a construção de subclasses.
- **RegistrableUser.java**: Classe abstrata que herda de `User`. É a base para utilizadores que necessitam de um registo mais detalhado (com NIF, morada, etc.), como `Employee` e `Client`.
- **Admin.java, Employee.java, Client.java**: Classes concretas que representam os diferentes papéis no sistema. Herdam de `User` ou `RegistrableUser` e adicionam atributos específicos do seu contexto (ex: `specialization` para `Employee`).
- **Repair.java**: Modela uma reparação, contendo informações como o código, estado, datas e custo.
- **Equipment.java, Part.java, Notification.java, Log.java**: Outras classes de modelo que representam as restantes entidades do sistema.

3.2.3. Package States

Este package é o coração da interface gráfica e da lógica de navegação. A aplicação utiliza o padrão de projeto State, onde cada "ecrã" é uma classe que herda de State. Isto permite isolar a lógica e a apresentação de cada vista.

- **State.java:** Classe abstrata base para todos os ecrãs. Gere a pilha de navegação (stateStack), permitindo avançar (next) e retroceder (back) entre ecrãs. Também gere a janela principal (JFrame) e o estado do utilizador autenticado.
 - buildView(): Método abstrato que as subclasses implementam para construir o seu JPanel (a sua vista).
 - enter(), next(State), back(): Métodos que controlam o ciclo de vida e a transição entre estados.
- **ListState<T>:** Classe abstrata genérica para ecrãs que apresentam uma lista de itens (ex: UserListState, RepairListState). Abstrai a lógica comum de buscar dados (fetchItems), construir uma tabela (JTable) e lidar com a seleção de um item (onSelect).
- **DetailState<T>:** Classe abstrata genérica para ecrãs que mostram os detalhes de um único item (ex: UserDetailState, RepairDetailState). Define uma estrutura para renderizar os campos (renderFields) e as ações disponíveis (getActions).
- **SignInState.java:** Implementa o ecrã de login. Valida as credenciais com o DatabaseManager e, em caso de sucesso, navega para o menu correspondente ao tipo de utilizador.
- **RepairDetailState.java:** Um exemplo de um estado complexo.
 - getActions(): Método que constrói dinamicamente a lista de botões de ação com base no tipo de utilizador (ADMIN, EMPLOYEE) e no estado atual da reparação (PENDING, COMPLETED, etc.). Esta é a lógica central que define o fluxo de trabalho das reparações.
 - statusButton(...): Método privado que cria um botão de ação. Ao ser clicado, confirma a ação com o utilizador e chama os métodos apropriados para atualizar o estado, como updateState() ou reject().

3.2.4. Package utils

Contém um conjunto de classes de utilidade (helpers) que realizam tarefas específicas e reutilizáveis em várias partes do projeto.

- **FormBuilder.java:** Uma classe muito útil que simplifica a criação de formulários com GridBagLayout, alinhando campos e botões de forma consistente e com menos código repetitivo.
- **FormValidator.java:** Centraliza a lógica de validação de formulários. Permite encadear várias regras de validação, incluindo verificações na base de dados (ex: dbValidate), e agrega todas as mensagens de erro para apresentar ao utilizador.

- **ImageService.java:** Gere o upload e download de imagens para um serviço externo. Abstrai a complexidade dos pedidos HTTP (POST para upload, GET para download).
- **Email.java:** Responsável pelo envio de emails através de SMTP (neste caso, configurado para o Gmail). É usado para notificar os utilizadores sobre o registo.
- **RepairStatementPrinter.java:** Gera um extrato em PDF para uma reparação, utilizando a biblioteca Apache PDFBox. Percorre os logs associados à reparação e formata-os num documento.

3.3. Algoritmos

Esta secção descreve os algoritmos mais importantes que constituem a base do funcionamento do projeto, desde a navegação e gestão de dados até à lógica de negócio específica.

3.3.1. Gestão de Estados e Navegação (Padrão de Projeto State)

O algoritmo central da aplicação é a gestão da interface gráfica através do padrão de projeto State. A navegação não é feita por um controlador central, mas sim por uma pilha de estados (Stack<State>) gerida na classe base State.java.

- **Funcionamento:**
 - Cada "ecrã" da aplicação é uma subclasse de State.
 - Quando se navega para um novo ecrã, o método `next(newState)` é chamado, o que resulta no push do novo estado para a pilha e na renderização da sua vista (`buildView`).
 - Para voltar atrás, o método `back()` faz pop do estado atual da pilha e renderiza o estado que ficou no topo.
- **Importância:** Este algoritmo desacopla completamente a lógica de cada ecrã, tornando o código mais modular e fácil de manter. A pilha de estados gere todo o fluxo de navegação de forma implícita.

```
public void enter() {
    stateStack.push(this);
    show();
}

public void next(State nextState) {
    nextState.enter();
}

public void back() {
    if (!stateStack.isEmpty()) stateStack.pop();
    if (!stateStack.isEmpty()) {
        stateStack.peek().show();
    }
}
```

3.3.2. Editar e Criar Users

A criação e atualização de utilizadores (saveUser e updateUser em DatabaseManager.java) é um algoritmo complexo devido ao modelo de dados hierárquico (User -> RegistrableUser -> Employee/Client).

- **Funcionamento (saveUser):**
 - Inicia uma transação na base de dados para garantir a atomicidade da operação.
 - Cria o registo base na tabela user.
 - Se o utilizador for do tipo RegistrableUser, cria um registo na tabela registrable_user e estabelece uma relação (RELATE) com o registo user.
 - Se for um Employee ou Client, repete o passo anterior para a tabela específica e relaciona-o com o registrable_user.
 - Se todos os passos forem bem-sucedidos, a transação é confirmada (commit). Caso contrário, é revertida (cancel), não deixando dados inconsistentes.
- **Importância:** Este algoritmo garante a integridade dos dados ao criar múltiplas entidades relacionadas como uma única operação indivisível.

```
public void saveUser(User user) {
    Transaction transaction = driver.beginTransaction();
    try {
        // Cria o registo base 'user'
        String query = "CREATE user CONTENT " + toSQL(User.toMap(user));
        Response response = transaction.query(query);
        user.setUserId(response.take(0).getArray().get(0).getObject().get
("id").getRecordId());

        if (user instanceof RegistrableUser) {
            // ... cria 'registrable_user' e relaciona com 'user'
            // ... se for Employee ou Client, cria o registo final e
            relaciona
        }
        transaction.commit(); // Confirma a transação
    } catch (Exception e) {
        transaction.cancel(); // Reverte em caso de erro
        // ...
    }
}
```

3.3.3. Geração Dinâmica de Ações no Fluxo de Reparação

O método `getActions()` na classe `RepairDetailState.java` implementa um algoritmo crucial para a lógica de negócio. Ele decide dinamicamente quais ações (botões) um utilizador pode realizar sobre uma reparação.

- **Funcionamento:**
 - algoritmo utiliza uma estrutura de switch dupla.
 - primeiro nível de decisão baseia-se no tipo de utilizador autenticado (ADMIN ou EMPLOYEE).
 - segundo nível baseia-se no estado atual da reparação (PENDING, IN_PROGRESS, COMPLETED, etc.).
 - Com base na combinação destes dois fatores, uma lista de botões (JButton) é construída e devolvida para ser renderizada na interface.
- **Importância:** Este algoritmo centraliza e implementa o fluxo de trabalho (workflow) das reparações, garantindo que os utilizadores só podem realizar as transições de estado permitidas para o seu papel e para o contexto atual.

```
protected List<JButton> getActions() {  
    List<JButton> actions = new ArrayList<>();  
    if (user.getType().equals("ADMIN")) {  
        switch (subject.getState()) {  
            case "PENDING":  
                actions.add(statusButton("Approve", "approve this repair",  
RepairStatus.ACCEPTED));  
                actions.add(statusButton("Reject", "reject this repair",  
RepairStatus.REJECTED_BY_ADMIN));  
                break;  
            // ... outros casos para ADMIN  
        }  
    } else if (user.getType().equals("EMPLOYEE")) {  
        switch (subject.getState()) {  
            case "ACCEPTED":  
                actions.add(statusButton("Accepted", "accepted this  
repair", RepairStatus.IN_PROGRESS));  
                // ... outros casos para EMPLOYEE  
                break;  
        }  
    }  
    return actions;  
}
```

3.3.4. Deserialização de Dados da Base de Dados

Os métodos *userFromValue* e *repairFromValue* em *DatabaseManager.java* são algoritmos de "tradução" essenciais. Eles convertem os dados brutos recebidos da base de dados (no formato *Value* da biblioteca *SurrealDB*) em objetos Java tipados do package *main.models*.

- **Funcionamento (*repairFromValue*):**
 - Recebe um objeto *Value* que representa um registo da tabela *repair*.
 - Para cada campo do modelo *Repair* (ex: *cost*, *observations*), o algoritmo primeiro verifica se o campo existe e não é nulo no objeto *Value* (*cost != null && cost.isNumber()*).
 - Apenas se a verificação for positiva é que o valor é extraído e atribuído ao objeto Java.
- **Importância:** Este algoritmo é fundamental para a robustez da aplicação, pois previne erros de *NullPointerException* ao lidar com campos que podem não existir em todos os registos (como o *cost* numa reparação ainda não concluída).

```
private Repair repairFromValue(Value element) {  
    var obj = element.getObject();  
    Repair repair = new Repair();  
    // ...  
    Value cost = obj.get("cost");  
    // ...  
    if (cost != null && !cost.isNull())  
        repair.setCost(cost.getDouble());  
    return repair;  
}
```

3.3.5. Validação de Formulários com Acesso à Base de Dados

A classe *FormValidator.java* implementa um algoritmo que combina validações de UI com validações na base de dados.

- **Funcionamento:**
 - Permite construir uma cadeia de validações (*require*, *dbValidate*).
 - método *dbValidate* recebe o campo, uma mensagem de erro e o nome de uma função da base de dados (ex: *fn::check_username*).
 - Ele invoca a função na base de dados através do *DatabaseManager* e, se receber uma mensagem de erro, adiciona-a à lista de erros do formulário.
 - No final, *isValid()* verifica se a lista de erros está vazia.
- **Importância:** Centraliza a lógica de validação, tornando os formulários mais limpos e garantindo que as regras de negócio (como a unicidade de um email) são verificadas diretamente na fonte de dados antes de qualquer tentativa de inserção.

```
private boolean validateFields() {
    FormValidator validator = new FormValidator()
        .require("Name", txtName)
        .dbValidate("Username", txtUsername, "fn::check_username")
        .require("Password", txtPassword)
        .dbValidate("Email", txtEmail, "fn::check_email");

    if (!type.equals("admin")) {
        validator
            .dbValidate("NIF", txtNif, "fn::check_nif")
            .dbValidate("Phone", txtPhone, "fn::check_phone")
            .require("Address", txtAddress);
    }
    if (type.equals("client")) {
        validator.require("Sector", txtSector);
    }
    if (!validator.isValid()) {
        JOptionPane.showMessageDialog(null, validator.getErrorMessage(),
"Error", JOptionPane.ERROR_MESSAGE);
        return false;
    }
    return true;
}
```

3.4. Estruturas de Dados

3.4.1. List e ArrayList

A estrutura de dados mais comum no projeto, utilizada para armazenar coleções de objetos obtidos da base de dados, como listas de utilizadores, reparações, equipamentos e notificações, e também para construir dinamicamente listas de componentes da interface gráfica, como os botões de ação nos ecrãs de detalhe. A ArrayList foi a implementação escolhida na maioria dos casos, justificada pela sua performance em operações de acesso e iteração. Como os dados são carregados da base de dados e depois maioritariamente lidos para preencher tabelas ou listas na UI, o acesso rápido por índice (`get(i)`) e a iteração eficiente da ArrayList são ideais. A sua capacidade de redimensionamento dinâmico também simplifica a gestão de coleções cujo tamanho não é conhecido à partida.

3.4.2. Map

Usada extensivamente para preparar os dados antes de serem enviados para a base de dados. As classes de modelo (como User, Repair, Equipment) contêm um método toMap() que converte o objeto num Map<String, Object>, utilizado depois para construir as queries de inserção e atualização, onde as chaves representam os nomes dos campos na base de dados e os valores são os dados a persistir. O Map é a estrutura natural para representar pares chave-valor, espelhando perfeitamente o formato de documento semelhante a JSON utilizado pela base de dados SurrealDB, desacoplando o modelo de dados da aplicação da sintaxe específica da base de dados. Dentro do projeto são usadas duas implementações: o HashMap, para casos onde a ordem não é relevante como no toMap() dos modelos, e o LinkedHashMap, nos ecrãs de detalhe para construir os menus de ações, garantindo que os botões são apresentados sempre na ordem em que foram definidos.

3.4.3. Stack

A estrutura de dados central que implementa o mecanismo de navegação entre os diferentes ecrãs da aplicação. A classe base State.java mantém uma pilha estática (stateStack) onde cada novo estado visitado é adicionado (push) e removido ao retroceder (pop). A escolha de uma Stack é ideal para gerir o histórico de navegação, uma vez que o seu comportamento LIFO (Last-In, First-Out) modela na perfeição a funcionalidade de retroceder: o último ecrã a ser visitado é o primeiro a ser removido. Esta foi uma decisão de arquitetura herdada do projeto 1 e mantida no projeto 2, onde a pilha continua a gerir a navegação mas agora entre JPanel dentro de uma JFrame em vez de ecrãs de texto.

3.4.4. Properties

Utilizada na classe PropertiesManager para carregar, manipular e guardar as configurações da aplicação, como as credenciais e o URL de ligação à base de dados, a partir do ficheiro config.properties. A classe java.util.Properties é a ferramenta padrão do Java para esta finalidade, sendo uma subclasse de Hashtable otimizada para armazenar pares de chave-valor do tipo String. Os métodos que disponibiliza (load, store, getProperty) abstraem toda a complexidade da leitura e escrita de ficheiros de propriedades, tornando a gestão de configurações simples e segura.

3.4.5. record

Para além das estruturas da Collections Framework, o projeto recorre também ao tipo record do Java, introduzido na versão 14. Usado para criar objetos imutáveis de transporte de dados como UserCredentials, DbFunctionCall e NotificationRequest, elimina a necessidade de escrever construtores, getters e métodos auxiliares manualmente. O record garante que estes objetos de transporte são sempre consistentes e imutáveis, o que reduz a possibilidade de erros e torna o código mais legível.

3.5. Armazenamento de Dados

A persistência de dados da aplicação é assegurada pela base de dados NoSQL SurrealDB. Ao contrário das bases de dados relacionais tradicionais, o SurrealDB é uma base de dados multi-modelo que suporta grafos, documentos e tabelas relacionais, permitindo tirar partido de relações entre registos de forma nativa através de edge tables (tabelas de ligação).

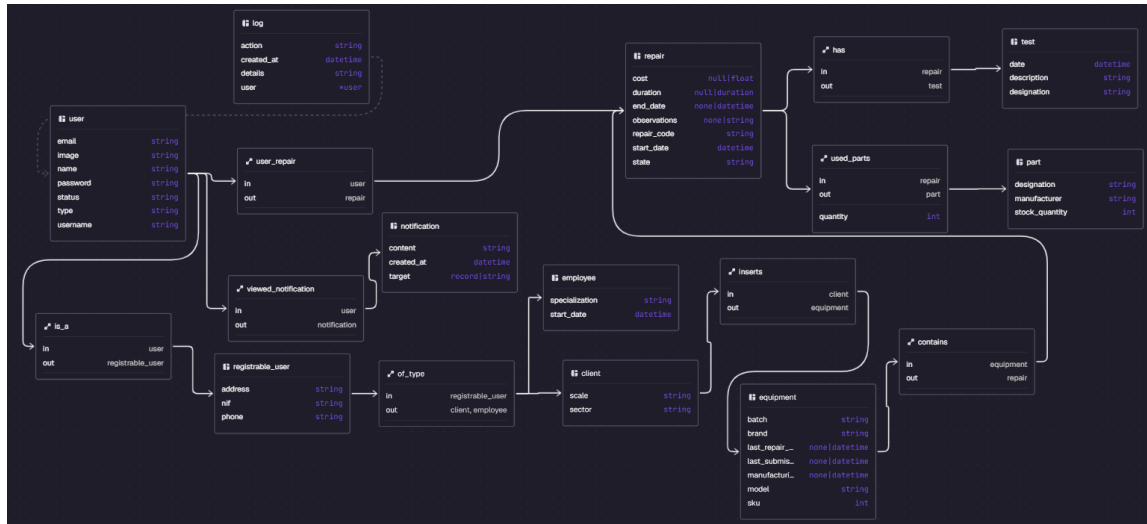


Figura 1 - Diagrama ER

A base de dados é composta pelas seguintes tabelas principais e respetivas relações:

Tabelas principais:

- **User**: armazena todos os utilizadores do sistema, com os campos name, username, password, email, status e type. O campo status pode assumir os valores PENDING, ACTIVE, INACTIVE e REJECTED. O campo type pode assumir os valores CLIENT, EMPLOYEE e ADMIN. O username é único e o email é validado através de uma assertion nativa do SurrealDB.
- **registrable_user**: armazena informação adicional comum a clientes e funcionários, nomeadamente nif, phone e address.
- **client**: armazena informação específica do cliente, nomeadamente scale (A, B, C ou D) e sector.
- **Employee**: armazena informação específica do funcionário, nomeadamente specialization (1 a 5) e start_date.
- **Equipment**: armazena os equipamentos dos clientes, com os campos brand, model, sku, batch, manufacturing_date, last_submission_date e last_repair_date. O campo sku é único e gerado aleatoriamente entre 1 e 1 000 000.
- **Repair**: armazena os pedidos de reparação, com os campos repair_code, state, start_date, end_date, duration, cost e observations. O state pode assumir os valores PENDING, ACCEPTED, IN_PROGRESS, COMPLETED, ARCHIVED, REJECTED_BY_ADMIN e REJECTED_BY_EMPLOYEE.

- **Test:** armazena os testes de operacionalidade realizados, com os campos designation, description e date.
- **Part:** armazena as peças disponíveis para reparação, com os campos designation, manufacturer e stock_quantity.
- **Log:** armazena o registo de ações dos utilizadores, com os campos action, created_at, details e user.
- **Notifications:** armazena as notificações enviadas aos utilizadores, com os campos content, created_at e target.

Tabelas de ligação (edge tables):

- **is_a:** liga user a registrable_user
- **of_type:** liga registrable_user a client ou employee
- **inserts:** liga client a equipment
- **contains:** liga equipment a repair
- **user_repair:** liga user a repair
- **has:** liga repair a test
- **used_parts:** liga repair a part, com o campo adicional quantity
- **viewed_notifications:** liga user a notifications

Triggers

Foram implementados triggers na base de dados para automatizar determinadas operações, garantindo a integridade dos dados e o registo automático de ações no log. Os triggers implementados são os seguintes:

Na tabela user:

- **log_user_create:** regista no log a criação de um novo utilizador
- **log_user_update:** regista no log as alterações efetuadas a um utilizador, detalhando os campos alterados

Na tabela inserts:

- **log_equipment_create:** regista no log a inserção de um novo equipamento, identificando o cliente responsável

Na tabela contains:

- **update_equipment_submission:** atualiza automaticamente o campo last_submission_date do equipment quando é submetido um novo pedido de reparação

Na tabela user_repair:

- **log_repair_create:** regista no log a criação de um novo pedido de reparação
- **log_repair_update:** regista no log as alterações de estado de uma reparação, identificando o utilizador responsável pela alteração

Na tabela repair:

- **repair_duration:** calcula e armazena automaticamente a duração da reparação quando o end_date é introduzido
- **repair_last_repair_date:** atualiza automaticamente o campo last_repair_date do equipment quando uma reparação passa ao estado IN_PROGRESS

Na tabela has:

- **log_test_create:** regista no log a adição de um teste de operacionalidade a uma reparação

Funções

Foram também implementadas funções nativas no SurrealDB para validação de dados e autenticação:

- **fn::check_email:** valida o formato do email
- **fn::check_nif:** valida o formato e unicidade do NIF
- **fn::check_phone:** valida o formato do contacto telefónico
- **fn::check_scale:** valida o escalão do cliente (A, B, C ou D)
- **fn::check_sku:** valida a unicidade do código SKU
- **fn::check_specialization:** valida a especialização do funcionário (1 a 5)
- **fn::check_username:** valida a unicidade do username
- **fn::get_user_type:** retorna o tipo de um utilizador dado o seu username
- **fn::user_exists:** verifica se as credenciais de autenticação são válidas
- **fn::generate_repair_code:** gera automaticamente o código único de uma reparação, composto pelo número sequencial de pedidos seguido da data e hora no formato AAAAMMDDHHMMSS. Por exemplo, se já existirem 95 reparações no sistema e for submetido um novo pedido às 15h33m10s do dia 19 de Fevereiro de 2026, o código gerado será 9620260219153310.

Ficheiro de Configuração

A ligação e configuração da aplicação é gerida através de um ficheiro Properties, que armazena todos os parâmetros necessários para o funcionamento do sistema. Este ficheiro contém três grupos de configurações distintos: os parâmetros de acesso à base de dados SurrealDB (endereço de ligação, namespace, nome da base de dados, username e password), as credenciais para o envio de emails via Gmail (endereço de email e chave de aplicação), e os parâmetros para o serviço de armazenamento de imagens (URL de upload, URL de download e token de autenticação). Este ficheiro pode ser alterado antes do arranque da aplicação, sendo que o ficheiro utilizado durante o desenvolvimento e testes se encontra disponível em anexo, permitindo ao avaliador executar a aplicação sem necessidade de configuração adicional, uma vez que inclui as credenciais de acesso à base de dados, ao serviço de email e ao serviço de armazenamento de imagens já configuradas.

3.6. Procedimentos de Teste

Os testes realizados ao longo do desenvolvimento do projeto foram efetuados de forma manual, através da execução da aplicação e verificação do comportamento esperado para cada funcionalidade implementada.

Para a base de dados, os testes foram realizados diretamente no SurrealDB Surrealist, executando queries e verificando os resultados obtidos, nomeadamente criação de utilizadores, inserção e validação de equipamentos, submissão e gestão de pedidos de reparação, verificação do correto funcionamento dos triggers e funções nativas, validação dos logs gerados automaticamente, e verificação das funções de validação de dados como NIF, telefone, email e SKU.

Para a aplicação Java, os testes foram realizados através da execução do programa e interação com a interface gráfica, verificando se as funcionalidades implementadas respondiam corretamente às ações do utilizador, se as mensagens de erro e sucesso eram apresentadas adequadamente, e se o envio de emails e o upload e visualização de fotografias de perfil funcionavam corretamente.

4. Conclusões

O desenvolvimento deste projeto permitiu consolidar os conhecimentos adquiridos ao longo da unidade curricular, aplicando-os na migração de uma aplicação de consola para uma interface gráfica completa com JavaSwing. Os objetivos propostos foram cumpridos na sua totalidade, resultando numa aplicação funcional, consistente e de fácil utilização.

4.1. Forças

A principal força deste projeto reside na arquitetura adotada, que permitiu reutilizar grande parte da lógica de negócio desenvolvida no projeto anterior sem alterações significativas. A separação clara entre a camada de dados (DatabaseManager), os modelos e a camada de apresentação (states) facilitou a migração para a interface gráfica, uma vez que apenas a camada de apresentação necessitou de ser reescrita.

A consistência do código é outro ponto forte, a definição de uma estrutura base (ListState, DetailState, FormBuilder) permitiu aplicar o mesmo padrão a todos os ecrãs da aplicação, tornando o código uniforme e fácil de manter, cumprindo o requisito G14.

A integração com serviços externos, nomeadamente o envio de emails via Jakarta Mail e o armazenamento de imagens num servidor remoto, acrescentou robustez à solução e garantiu que as fotografias de perfil funcionam corretamente independentemente da máquina onde a aplicação é executada

4.2. Limitações

A principal limitação identificada prende-se com o sistema de envio de emails. A solução implementada requer que a conta Gmail utilizada tenha autenticação de dois fatores ativa e que seja gerada uma palavra-passe de aplicação específica. Esta configuração, embora funcional no ambiente de testes, não é transparente para o utilizador final e limita a flexibilidade da solução, qualquer alteração à conta de email implica atualizar manualmente o ficheiro de configuração.

Por fim, a aplicação depende de uma ligação constante ao servidor onde corre o SurrealDB, não existindo qualquer mecanismo de funcionamento offline ou de cache local de dados.

4.3. Trabalho Futuro

A adição de testes automatizados, recorrendo a frameworks como JUnit, aumentaria a robustez da aplicação e facilitaria a deteção de erros em desenvolvimentos futuros.

Seria também relevante melhorar a interface gráfica com temas personalizáveis e uma experiência de utilizador mais moderna, eventualmente migrando para JavaFX que oferece mais flexibilidade de design do que o JavaSwing.

5. Referências

- baeldung. “Baeldung.” *Baeldung on Kotlin*, 24 Jan. 2024, www.baeldung.com/java-builder-pattern-inheritance. Accessed 18 Apr. 2026.
- Eckel, Bruce. “Curiously Recurring Generic Pattern.” *Artima.com*, 2019, www.artima.com/weblogs/viewpost.jsp?thread=133275. Accessed 18 Apr. 2026.
- Gomes, Robson Fernando. “Utilizando Arquivos de Propriedades No Java.” *DevMedia*, 24 Aug. 2012, www.devmedia.com.br/utilizando-arquivos-de-propriedades-no-java/25546. Accessed 18 Apr. 2026.
- Java Fusion. “What Is a .Properties File in Java.” *Medium*, 28 Jan. 2024, medium.com/@JavaFusion/what-is-a-properties-file-in-java-e955c3adc92f. Accessed 18 Apr. 2026.
- “Properties (the Java™ Tutorials > Essential Java Classes > the Platform Environment).” *Docs.oracle.com*, 2024, docs.oracle.com/javase/tutorial/essential/environment/properties.html. Accessed 18 Apr. 2026.
- surrealDB. “Java SDK | SurrealDB Docs.” *Surrealdb.com*, 2026, surrealdb.com/docs/languages/java. Accessed 18 Apr. 2026.
- Wendel Segadilha. “Como Ler Arquivos .Properties No Java.” *YouTube*, 6 Mar. 2024, www.youtube.com/watch?v=jRigxelb43E. Accessed 18 Apr. 2026.
- Y-N, Ken. “Subclassing a Java Builder Class.” *Stack Overflow*, 18 June 2013, stackoverflow.com/questions/17164375/subclassing-a-java-builder-class. Accessed 18 Apr. 2026.

“Pdfbox 3.0.3 Javadoc (Org.apache.pdfbox).” *Javadoc.io*, 2026,
javadoc.io/doc/org.apache.pdfbox/pdfbox/3.0.3/index.html. Accessed 27 June
2026.

to, JLabel. “Clicking a JLabel to Open a New Frame.” *Stack Overflow*, 6 Feb. 2013,
stackoverflow.com/questions/14735085/clicking-a-jlabel-to-open-a-new-frame.
Accessed 17 June 2026.

Vincent Aliquo. “Java How To: Clickable JLabels.” *YouTube*, 3 Nov. 2015,
www.youtube.com/watch?v=FY3uOvullIO0. Accessed 17 June 2026.

6. Anexos

Github:

https://github.com/Marycrazy/LEI_PA_2026_AP1_Mariana-Magalhaes_Dinis-Myroshnyk2026-03-30

Diagrama e requisitos:

[Projeto 2 PA](#)